UNIVERSIDAD CATÓLICA DE SANTA MARÍA

ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS

LABORATORIO N.° 8

BASES DE DATOS SQLITE Y JDBC EN JAVA

CASO D: PLATAFORMA DE JUEGO MULTIJUGADOR

Curso: Lenguajes de Programación III

Estudiante: Gian Piero Khalil Rodríguez Fádel

Práctica: Sesión N.° 08

AREQUIPA – PERÚ

Octubre de 2026

ÍNDICE

1. Resumen y objetivos

2. Planteamiento del caso

3. Diseño de base de datos

4. Desarrollo de los ejercicios

5. Comprobaciones y resultados

6. Cuestionario

7. Conclusiones

8. Bibliografía

Anexo. Código fuente y simulación de ejecución por archivo

1. RESUMEN Y OBJETIVOS

Se desarrolló un sistema de consola en Java para gestionar partidas, usuarios, juegos, perfiles, participaciones, estadísticas y logros de una plataforma multijugador. La información se almacena en SQLite a través de JDBC. Se respetó el esquema construido durante las primeras etapas de la práctica y se ampliaron las operaciones para cumplir los ejercicios de manipulación de datos.

Objetivo general. Implementar una aplicación Java que administre una base de datos relacional SQLite de ocho tablas y demuestre operaciones CRUD, consultas parametrizadas, transacciones y procesamiento de registros en memoria.

Objetivos específicos. Crear claves primarias y foráneas; insertar diez registros de prueba en cada tabla; ejecutar siete consultas con condiciones; aplicar tres actualizaciones y tres eliminaciones; confirmar o revertir cambios mediante contraseña; realizar filtros desde un arreglo de objetos en Java.

2. PLANTEAMIENTO DEL CASO

Una plataforma multijugador permite que una persona se registre, mantenga un perfil, participe en diversas partidas, consulte su desempeño acumulado y desbloquee logros. Un juego puede disponer de múltiples partidas, y una partida admite varios usuarios. La relación de usuarios con partidas es muchos a muchos porque se conserva el historial de participación, incluso cuando solo se permite una partida activa al mismo tiempo.

La regla de una sola partida activa por jugador se comprueba desde el menú Java al registrar una participación. Para una plataforma con varios clientes concurrentes sería recomendable reforzarla también mediante controles transaccionales de la base de datos.

3. DISEÑO DE BASE DE DATOS

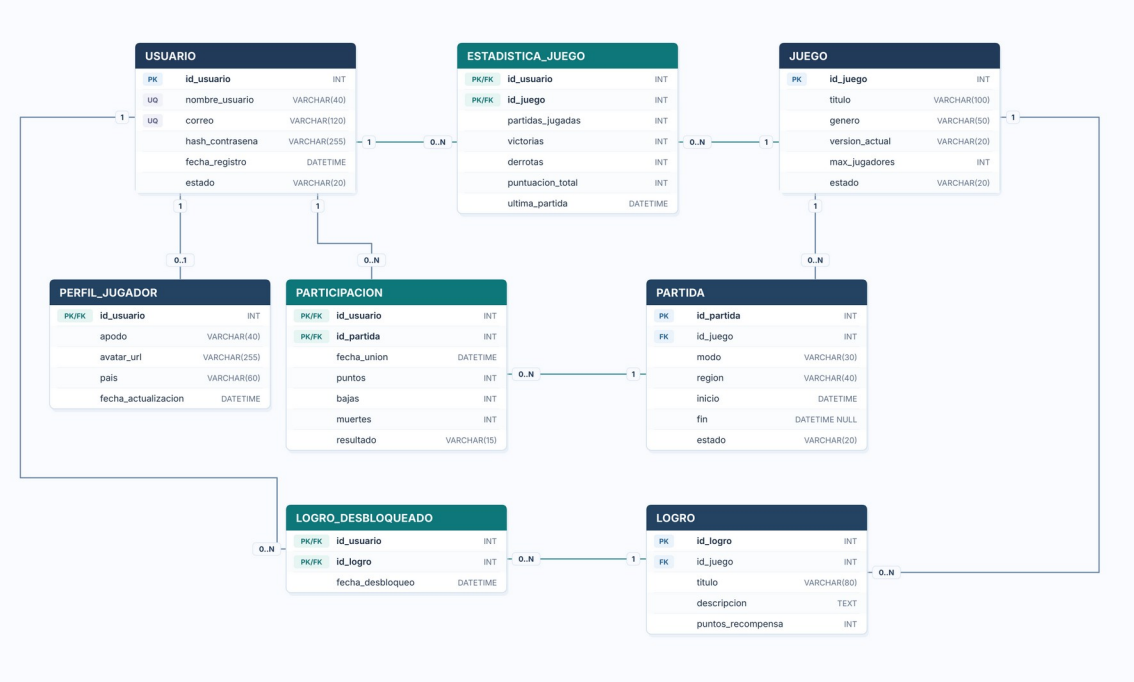


Figura 1. Modelo entidad–relación utilizado para desarrollar las ocho tablas.

Tabla	Clave primaria	Relación principal
usuario	id_usuario	1:1 con perfil_jugador
perfil_jugador	id_usuario	FK hacia usuario
juego	id_juego	1:N con partida y logro
partida	id_partida	FK hacia juego
participacion	id_usuario, id_partida	N:M usuario–partida
estadistica_juego	id_usuario, id_juego	N:M usuario–juego
logro	id_logro	FK hacia juego
logro_desbloqueado	id_usuario, id_logro	N:M usuario–logro

Las claves foráneas se habilitan en cada conexión con PRAGMA foreign_keys = ON. La combinación de clave primaria y clave foránea en perfil_jugador impide que un usuario tenga más de un perfil. Las tablas intermedias emplean claves compuestas para evitar asociaciones duplicadas.

4. DESARROLLO DE LOS EJERCICIOS

4.1. Creación de tablas

La clase BaseDeDatos mantiene las ocho sentencias CREATE TABLE IF NOT EXISTS. Se utiliza Statement porque se trata de instrucciones DDL y se abre la conexión con DriverManager.getConnection(). No se modifica el esquema existente si una tabla ya estaba creada; para cambios estructurales posteriores sería necesaria una migración.

4.2. Inserción de registros

DatosPrueba utiliza consultas INSERT OR IGNORE con parámetros ?, establecidos desde PreparedStatement. La carga contiene al menos diez filas por entidad y sigue el orden exigido por las claves foráneas. Los campos autogenerados, como fechas de registro, se omiten cuando tienen un valor DEFAULT.

4.3. Consultas SQL

N.º	Consulta	Condición
1	Usuarios activos	estado = ACTIVO
2	Juegos por capacidad	max_jugadores >= valor
3	Partidas por videojuego	id_juego = valor
4	Historial de un jugador	id_usuario = valor
5	Puntajes de participaciones	puntos >= valor
6	Partidas finalizadas por región	estado = FINALIZADA y región
7	Logros de un jugador	id_usuario = valor

Las consultas parametrizan los valores mediante PreparedStatement y muestran los resultados leyendo un ResultSet. En las consultas de historial y logros se unen varias tablas con JOIN, sin duplicar innecesariamente información.

4.4. Tres actualizaciones y tres eliminaciones

La clase EjerciciosSQL modifica el estado de un usuario, la versión de un juego y el país de un perfil. Después elimina una relación de logro desbloqueado, una participación y una estadística. Los seis cambios utilizan WHERE y se realizan en una transacción: commit() los conserva y rollback() los revierte.

4.5. Aplicación CRUD de consola

El menú permite elegir cualquiera de las ocho tablas y realizar las cuatro operaciones básicas. GestorCRUD obtiene los campos mediante PRAGMA table_info; con esta información puede gestionar claves primarias sencillas o compuestas. Los valores se reciben desde Scanner y se

asignan como parámetros a las sentencias SQL, mientras los nombres de tabla se validan con una lista blanca fija.

4.6. Confirmación de cambios

Cada operación que cambia información solicita una clave antes de ejecutar commit(). Si la clave no coincide, se utiliza rollback(). La clave didáctica predeterminada es lab8 y puede cambiarse con LAB8_CLAVE. No debe utilizarse como seguridad de un sistema real.

4.7. Clase gestora sobre arreglo de objetos

GestorRegistros recupera información una sola vez, construye objetos Registro y los guarda en un arreglo Registro[]. Luego se ejecutan en Java las condiciones de comparación, la selección de columnas, el orden ascendente o descendente y la limitación del número de registros. Esta operación muestra la diferencia entre filtrar con SQL y filtrar información en memoria.

5. COMPROBACIONES Y RESULTADOS

La compilación de las clases Java finalizó sin errores con javac (Java 21). No se realizó una ejecución JDBC completa en el entorno de elaboración porque no se disponía localmente del archivo sqlite-jdbc requerido. Por ello, las figuras de consola se presentan con la indicación expresa SIMULACIÓN. La ejecución SQL sí fue comprobada con SQLite mediante un script de verificación que reproduce el esquema, la carga inicial, las siete consultas y las transacciones.

Prueba	Resultado comprobado
Creación de tablas	8 tablas creadas
Inserciones	10 registros iniciales en cada tabla
Consultas con condiciones	7 consultas con filas recuperadas
Relación 1:1	Se rechazó un perfil duplicado
Clave foránea	Se rechazó usuario inexistente
Rollback	El cambio no quedó guardado
3 UPDATE + 3 DELETE	6 operaciones afectaron una fila cada una
Integridad posterior	PRAGMA foreign_key_check sin errores

```
TABLAS CREADAS: 8
estadistica_juego     : 10 registros
juego                 : 10 registros
logro                 : 10 registros
logro_desbloqueado    : 10 registros
participacion         : 10 registros
partida               : 10 registros
perfil_jugador        : 10 registros
usuario               : 10 registros
CONSULTA usuarios_activos    : 10 filas
CONSULTA juegos_capacidad    : 10 filas
CONSULTA partidas_por_juego  : 2 filas
CONSULTA historial_usuario   : 2 filas
CONSULTA puntaje_minimo      : 5 filas
CONSULTA finalizadas_region  : 5 filas
```

```
CONSULTA logros_usuario      : 2 filas
PRUEBA 1:1: duplicado rechazado
PRUEBA FK: usuario inexistente rechazado
PRUEBA ROLLBACK: cambio revertido correctamente
PRUEBA 3 UPDATE + 3 DELETE: seis operaciones confirmadas
PRUEBA FINAL FK: sin referencias invalidas
RESULTADO: VALIDACION SQL SUPERADA
```

6. CUESTIONARIO

1. ¿Qué es SQLite y cuáles son sus características?

Es un motor relacional embebido que guarda la base en un archivo, no requiere un servidor dedicado y permite usar transacciones y SQL. En este trabajo el archivo es multijugador.db.

2. ¿Qué es JDBC y cómo se utiliza?

JDBC es la API de Java para interactuar con bases de datos. Se emplean Connection, PreparedStatement y ResultSet, junto al controlador sqlite-jdbc para SQLite.

3. ¿Para qué sirve DriverManager?

DriverManager permite obtener conexiones mediante una URL JDBC y los controladores disponibles. Nuestra aplicación utiliza DriverManager.getConnection(URL).

4. ¿Cómo se obtiene una conexión con SQLite?

Se usa la URL jdbc:sqlite:multijugador.db; basta esa cadena para abrir o crear el archivo de base de datos, siempre que el controlador esté disponible.

5. ¿Qué es una sentencia SQL y cómo se ejecuta?

Es una instrucción de definición o manipulación de datos, como CREATE TABLE, INSERT, UPDATE o SELECT. En Java se ejecuta mediante Statement o PreparedStatement.

6. ¿Qué diferencia hay entre Statement y PreparedStatement?

Statement ejecuta una sentencia directa. PreparedStatement permite parámetros ?, separando valores de la estructura SQL y facilitando la reutilización de la consulta.

7. ¿Cuándo conviene PreparedStatement y por qué?

Cuando los datos proceden del usuario o cambian entre ejecuciones, porque parametriza valores y evita que esos valores se interpreten como instrucciones SQL.

8. ¿Cómo ejecutar SELECT y recuperar datos?

Se obtiene una conexión, se prepara el SELECT, se asignan parámetros y se utiliza executeQuery(). Un ciclo while(rs.next()) recorre cada registro del ResultSet.

9. ¿Cómo administrar transacciones?

Se desactiva autoCommit, se realizan las operaciones y se llama commit si deben conservarse o rollback si deben descartarse. Es necesario gestionar errores y cerrar la conexión.

10. ¿Qué hace executeUpdate()?

Ejecuta INSERT, UPDATE, DELETE y determinadas instrucciones DDL; devuelve un entero de filas afectadas en operaciones DML.

11. ¿Cómo manejar excepciones JDBC?

Se captura SQLException, se informa del problema, se ejecuta rollback si hay cambios sin confirmar y se liberan los recursos con try-with-resources.

12. ¿Cómo se utilizan las consultas parametrizadas?

Se escribe una consulta con ?, se llama prepareStatement y luego setInt, setString o setObject para asignar cada valor según su posición.

13. ¿Por qué cerrar Connection, Statement y ResultSet?

Porque consumen recursos y conexiones abiertas pueden mantener bloqueos. try-with-resources permite cerrarlos incluso si ocurre una excepción.

7. CONCLUSIONES

La práctica permitió comprobar cómo un modelo entidad–relación se convierte en un conjunto de tablas SQLite con restricciones reales. Las claves compuestas resolvieron las relaciones de muchos a muchos y las claves foráneas evitaron registros huérfanos.

Separar la conexión, la carga de datos, el CRUD y las consultas simplificó la aplicación. PreparedStatement evitó concatenar los valores introducidos por el usuario y las transacciones permitieron aceptar o rechazar varias modificaciones como una sola unidad de trabajo.

La clase gestora completó el aprendizaje al realizar operaciones similares a SELECT directamente sobre un arreglo de objetos Java. Queda como mejora futura incorporar autenticación segura y pruebas de concurrencia reales para partidas activas.

8. BIBLIOGRAFÍA

Oracle. (s. f.). JDBC basics. Java Tutorials. https://docs.oracle.com/javase/tutorial/jdbc/basics/

SQLite. (s. f.). SQLite foreign key support. https://www.sqlite.org/foreignkeys.html

SQLite. (s. f.). Transaction. https://www.sqlite.org/lang_transaction.html

SQLite. (s. f.). CREATE TABLE. https://www.sqlite.org/lang_createtable.html

Xerial. (2026). SQLite JDBC driver (versión 3.53.4.0). GitHub. https://github.com/xerial/sqlite-jdbc

Universidad Católica de Santa María. (2026). Guía de prácticas de Lenguajes de Programación III: Sesión N.° 08 [Material de clase].

ANEXO. ARCHIVOS JAVA, EXPLICACIÓN Y EVIDENCIAS SIMULADAS

Se incluye el código de los nueve archivos Java del proyecto, junto a una descripción concreta de su función y una imagen ilustrativa de consola para cada archivo. Estas figuras fueron construidas con base en el comportamiento previsto y no constituyen capturas de ejecución real en Eclipse.

A.1. conexion/BaseDeDatos.java

Conserva el trabajo inicial del laboratorio: establece la conexión con SQLite, activa las claves foráneas y crea las ocho entidades. No se reemplazó la implementación del estudiante. La misma estructura admite volver a iniciar el programa sin intentar duplicar tablas.

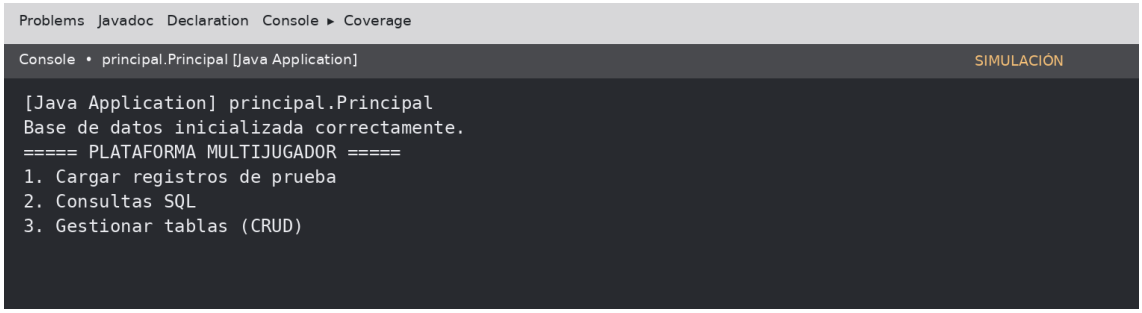


Figura A.1. Consola de Eclipse — simulación de uso relacionada con BaseDeDatos.java.

Código fuente completo:

```
package conexion;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.sql.Statement;

public class BaseDeDatos {
    private static final String URL = "jdbc:sqlite:multijugador.db";

    public static Connection getConnection() throws SQLException
    {
        Connection con = DriverManager.getConnection(URL);

        try
        {
            try(Statement stmt = con.createStatement())
            {
                stmt.execute("PRAGMA foreign_keys = ON");
            }
            return con;
        }
        catch(SQLException e)
        {
            con.close();
            throw e;
        }
    }

    public static void inicializar() throws SQLException
    {
        try(Connection con = getConnection(); Statement stmt = con.createStatement())
        {
            String sqlUsuario = """
                    CREATE TABLE IF NOT EXISTS usuario
                    (
                        id_usuario INTEGER PRIMARY KEY,
                        nombre_usuario TEXT NOT NULL UNIQUE,
                        correo TEXT NOT NULL UNIQUE,
                        hash_contrasena TEXT NOT NULL,
                        fecha_registro TEXT DEFAULT CURRENT_TIMESTAMP,
                        estado TEXT NOT NULL
                    )
                    """;
            stmt.execute(sqlUsuario);

            String sqlPerfil = """
```

```
        CREATE TABLE IF NOT EXISTS perfil_jugador
        (
            id_usuario INTEGER PRIMARY KEY,
            apodo TEXT,
            avatar_url TEXT,
            pais TEXT,
            fecha_actualizacion DEFAULT CURRENT_TIMESTAMP,

            FOREIGN KEY (id_usuario) REFERENCES usuario(id_usuario)
        )
        """;
stmt.execute(sqlPerfil);

String sqlJuego = """
        CREATE TABLE IF NOT EXISTS juego
        (
            id_juego INTEGER PRIMARY KEY,
            titulo TEXT NOT NULL,
            genero TEXT,
            version_actual TEXT NOT NULL,
            max_jugadores INTEGER NOT NULL,
            estado TEXT NOT NULL
        )
        """;
stmt.execute(sqlJuego);

String sqlPartida = """
        CREATE TABLE IF NOT EXISTS partida
        (
            id_partida INTEGER PRIMARY KEY,
            id_juego INTEGER NOT NULL,
            modo TEXT NOT NULL,
            region TEXT NOT NULL,
            inicio TEXT DEFAULT CURRENT_TIMESTAMP NOT NULL,
            fin TEXT,
            estado TEXT NOT NULL,

            FOREIGN KEY (id_juego) REFERENCES juego(id_juego)
        )
        """;
stmt.execute(sqlPartida);

String sqlParticipacion = """
        CREATE TABLE IF NOT EXISTS participacion
        (
            id_usuario INTEGER NOT NULL,
            id_partida INTEGER NOT NULL,
            fecha_union TEXT DEFAULT CURRENT_TIMESTAMP,
            puntos INTEGER NOT NULL DEFAULT 0,
            bajas INTEGER NOT NULL DEFAULT 0,
            muertes INTEGER NOT NULL DEFAULT 0,
            resultado TEXT,

            PRIMARY KEY(id_usuario, id_partida),

            FOREIGN KEY (id_usuario) REFERENCES usuario(id_usuario),
            FOREIGN KEY (id_partida) REFERENCES partida(id_partida)
        )
        """;
stmt.execute(sqlParticipacion);

String sqlEstadistica = """
        CREATE TABLE IF NOT EXISTS estadistica_juego
        (
            id_usuario INTEGER NOT NULL,
            id_juego INTEGER NOT NULL,
            partidas_jugadas INTEGER NOT NULL DEFAULT 0,
            victorias INTEGER NOT NULL DEFAULT 0,
            derrotas INTEGER NOT NULL DEFAULT 0,
            puntuacion_total INTEGER NOT NULL DEFAULT 0,
            ultima_partida TEXT,

            PRIMARY KEY (id_usuario, id_juego),

            FOREIGN KEY (id_usuario) REFERENCES usuario(id_usuario),
```

```
                    FOREIGN KEY (id_juego) REFERENCES juego(id_juego)
                )
                """;
            stmt.execute(sqlEstadistica);

            String sqlLogro = """
                    CREATE TABLE IF NOT EXISTS logro
                    (
                        id_logro INTEGER PRIMARY KEY,
                        id_juego INTEGER NOT NULL,
                        titulo TEXT NOT NULL,
                        descripcion TEXT,
                        puntos_recompensa INTEGER NOT NULL DEFAULT 0,

                        FOREIGN KEY (id_juego) REFERENCES juego(id_juego)
                    )
                    """;
            stmt.execute(sqlLogro);

            String sqlLogroDesbloqueado = """
                    CREATE TABLE IF NOT EXISTS logro_desbloqueado
                    (
                        id_usuario INTEGER NOT NULL,
                        id_logro INTEGER NOT NULL,
                        fecha_desbloqueo TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,

                        PRIMARY KEY (id_usuario, id_logro),

                        FOREIGN KEY (id_usuario) REFERENCES usuario(id_usuario),
                        FOREIGN KEY (id_logro) REFERENCES logro(id_logro)
                    )
                    """;
            stmt.execute(sqlLogroDesbloqueado);
        }
    }
}
```

A.2. conexion/DatosPrueba.java

Inserta diez registros por tabla respetando el orden de las relaciones. Usa PreparedStatement para parametrizar los valores y INSERT OR IGNORE para que una segunda ejecución no duplique los datos de prueba. La transacción se controla desde el menú principal.

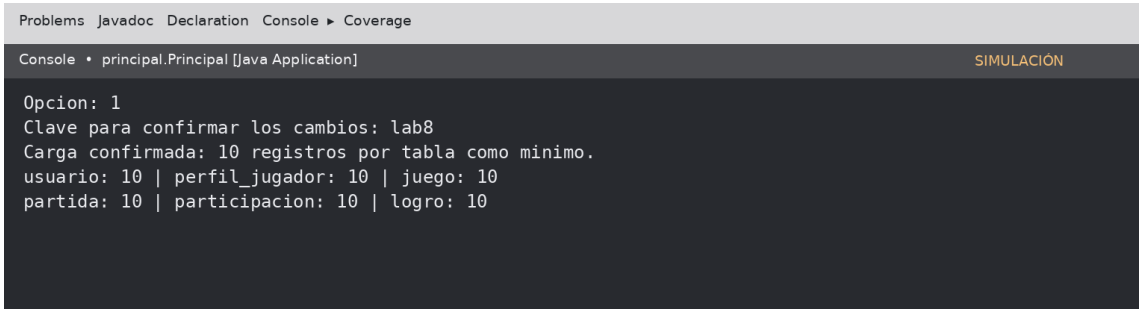


Figura A.2. Consola de Eclipse — simulación de uso relacionada con DatosPrueba.java.

Código fuente completo:

```
package conexion;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.SQLException;

public class DatosPrueba {
    public static void cargar(Connection con) throws SQLException
    {
        insertar(con, "INSERT OR IGNORE INTO usuario (id_usuario, nombre_usuario, correo,
hash_contrasena, estado) VALUES (?, ?, ?, ?, ?)", new Object[][] {
                {1, "Piero", "piero@ejemplo.com", "HASH_DE_PRUEBA_1", "ACTIVO"},
                {2, "Andrea", "andrea@ejemplo.com", "HASH_DE_PRUEBA_2", "ACTIVO"},
                {3, "Carlos", "carlos@ejemplo.com", "HASH_DE_PRUEBA_3", "ACTIVO"},
                {4, "Lucia", "lucia@ejemplo.com", "HASH_DE_PRUEBA_4", "ACTIVO"},
                {5, "Diego", "diego@ejemplo.com", "HASH_DE_PRUEBA_5", "ACTIVO"},
                {6, "Camila", "camila@ejemplo.com", "HASH_DE_PRUEBA_6", "ACTIVO"},
                {7, "Mateo", "mateo@ejemplo.com", "HASH_DE_PRUEBA_7", "ACTIVO"},
                {8, "Valeria", "valeria@ejemplo.com", "HASH_DE_PRUEBA_8", "ACTIVO"},
                {9, "Sofia", "sofia@ejemplo.com", "HASH_DE_PRUEBA_9", "ACTIVO"},
                {10, "Bruno", "bruno@ejemplo.com", "HASH_DE_PRUEBA_10", "ACTIVO"}
        });

        insertar(con, "INSERT OR IGNORE INTO perfil_jugador (id_usuario, apodo, avatar_url, pais)
VALUES (?, ?, ?, ?)", new Object[][] {
                {1, "PieroX", "avatars/1.png", "Peru"}, {2, "Andromeda", "avatars/2.png", "Peru"},
                {3, "CarlosGG", "avatars/3.png", "Chile"}, {4, "Luci4", "avatars/4.png", "Peru"},
                {5, "D_Rex", "avatars/5.png", "Mexico"}, {6, "Camix", "avatars/6.png", "Peru"},
                {7, "Mateo7", "avatars/7.png", "Argentina"}, {8, "Valkyrie", "avatars/8.png",
"Peru"},
                {9, "SofiStar", "avatars/9.png", "Chile"}, {10, "BRun", "avatars/10.png", "Colombia"}
        });

        insertar(con, "INSERT OR IGNORE INTO juego (id_juego, titulo, genero, version_actual,
max_jugadores, estado) VALUES (?, ?, ?, ?, ?, ?)", new Object[][] {
                {1, "Arena Nova", "Accion", "1.2", 10, "ACTIVO"},
                {2, "Reino Pixel", "RPG", "2.0", 8, "ACTIVO"},
                {3, "Velocidad X", "Carreras", "1.5", 12, "ACTIVO"},
                {4, "Titan Clash", "Estrategia", "3.1", 6, "ACTIVO"},
                {5, "Galaxia Sur", "Aventura", "1.7", 4, "ACTIVO"},
                {6, "Guardianes", "Accion", "1.1", 10, "ACTIVO"},
                {7, "Cubo Magico", "Puzzle", "2.2", 4, "ACTIVO"},
                {8, "Mundo Mar", "Supervivencia", "1.3", 6, "ACTIVO"},
                {9, "Zona Roja", "Accion", "1.0", 12, "ACTIVO"},
                {10, "Fortaleza", "Estrategia", "2.5", 8, "ACTIVO"}
        });
```

```
        insertar(con, "INSERT OR IGNORE INTO partida (id_partida, id_juego, modo, region, inicio,
fin, estado) VALUES (?, ?, ?, ?, ?, ?, ?)", new Object[][] {
                {101, 1, "Equipos", "LATAM", "2026-10-18 18:00:00", "2026-10-18 18:35:00",
"FINALIZADA"},
                {102, 1, "Equipos", "LATAM", "2026-10-19 19:00:00", "2026-10-19 19:40:00",
"FINALIZADA"},
                {103, 2, "Cooperativo", "LATAM", "2026-10-19 20:00:00", "2026-10-19 20:30:00",
"FINALIZADA"},
                {104, 2, "Duelo", "EU", "2026-10-19 21:00:00", "2026-10-19 21:20:00", "FINALIZADA"},
                {105, 3, "Carrera", "LATAM", "2026-10-20 16:00:00", "2026-10-20 16:15:00",
"FINALIZADA"},
                {106, 3, "Carrera", "NA", "2026-10-20 16:20:00", "2026-10-20 16:40:00",
"FINALIZADA"},
                {107, 4, "Equipos", "LATAM", "2026-10-20 17:00:00", "2026-10-20 17:25:00",
"FINALIZADA"},
                {108, 4, "Duelo", "EU", "2026-10-20 18:00:00", "2026-10-20 18:20:00", "FINALIZADA"},
                {109, 5, "Cooperativo", "LATAM", "2026-10-21 21:00:00", null, "EN_CURSO"},
                {110, 5, "Cooperativo", "LATAM", "2026-10-21 21:30:00", null, "ESPERA"}
        });

        insertar(con, "INSERT OR IGNORE INTO participacion (id_usuario, id_partida, puntos, bajas,
muertes, resultado) VALUES (?, ?, ?, ?, ?, ?)", new Object[][] {
                {1, 101, 170, 14, 6, "VICTORIA"}, {2, 101, 90, 8, 9, "DERROTA"},
                {1, 102, 120, 11, 8, "VICTORIA"}, {3, 102, 80, 7, 10, "DERROTA"},
                {4, 103, 150, 12, 3, "VICTORIA"}, {5, 104, 50, 3, 12, "DERROTA"},
                {6, 105, 200, 0, 0, "VICTORIA"}, {7, 106, 70, 0, 0, "DERROTA"},
                {8, 107, 140, 8, 5, "VICTORIA"}, {9, 109, 0, 0, 0, null}
        });

        insertar(con, "INSERT OR IGNORE INTO estadistica_juego (id_usuario, id_juego,
partidas_jugadas, victorias, derrotas, puntuacion_total, ultima_partida) VALUES
(?, ?, ?, ?, ?, ?, ?)", new Object[][] {
                {1, 1, 2, 2, 0, 290, "2026-10-19 19:40:00"},
                {2, 1, 1, 0, 1, 90, "2026-10-18 18:35:00"},
                {3, 1, 1, 0, 1, 80, "2026-10-19 19:40:00"},
                {4, 2, 1, 1, 0, 150, "2026-10-19 20:30:00"},
                {5, 2, 1, 0, 1, 50, "2026-10-19 21:20:00"},
                {6, 3, 1, 1, 0, 200, "2026-10-20 16:15:00"},
                {7, 3, 1, 0, 1, 70, "2026-10-20 16:40:00"},
                {8, 4, 1, 1, 0, 140, "2026-10-20 17:25:00"},
                {9, 5, 0, 0, 0, 0, null},
                {10, 5, 0, 0, 0, 0, null}
        });

        insertar(con, "INSERT OR IGNORE INTO logro (id_logro, id_juego, titulo, descripcion,
puntos_recompensa) VALUES (?, ?, ?, ?, ?)", new Object[][] {
                {1, 1, "Primera partida", "Disputa una partida", 100},
                {2, 1, "Dos victorias", "Gana dos partidas", 200},
                {3, 2, "Explorador", "Descubre un area", 50},
                {4, 2, "Maestro", "Supera una mision", 150},
                {5, 3, "Rapido", "Gana una carrera", 100},
                {6, 3, "Piloto", "Participa en una carrera", 300},
                {7, 4, "Estratega", "Gana un duelo", 100},
                {8, 4, "Comandante", "Gana cinco duelos", 250},
                {9, 5, "Pionero", "Inicia una aventura", 50},
                {10, 5, "Leyenda", "Completa la aventura", 400}
        });

        insertar(con, "INSERT OR IGNORE INTO logro_desbloqueado (id_usuario, id_logro,
fecha_desbloqueo) VALUES (?, ?, ?)", new Object[][] {
                {1, 1, "2026-10-18 18:35:00"}, {1, 2, "2026-10-19 19:40:00"},
                {2, 1, "2026-10-18 18:35:00"}, {3, 1, "2026-10-19 19:40:00"},
                {4, 3, "2026-10-19 20:30:00"}, {5, 4, "2026-10-19 21:20:00"},
                {6, 5, "2026-10-20 16:15:00"}, {7, 6, "2026-10-20 16:40:00"},
                {8, 7, "2026-10-20 17:25:00"}, {9, 9, "2026-10-21 21:00:00"}
        });
    }

    private static void insertar(Connection con, String sql, Object[][] registros) throws
SQLException
    {
        try(PreparedStatement ps = con.prepareStatement(sql))
        {
            for(Object[] registro : registros)
            {
```

```
                for(int i = 0; i < registro.length; i++)
                {
                    ps.setObject(i + 1, registro[i]);
                }
                ps.executeUpdate();
            }
        }
    }
}
```

A.3. consultas/Consultas.java

Reúne siete consultas con condiciones definidas: usuarios activos, capacidad mínima, partidas por juego, historial de usuario, puntajes mínimos, partidas finalizadas por región y logros desbloqueados. Varias utilizan JOIN, WHERE y ORDER BY.

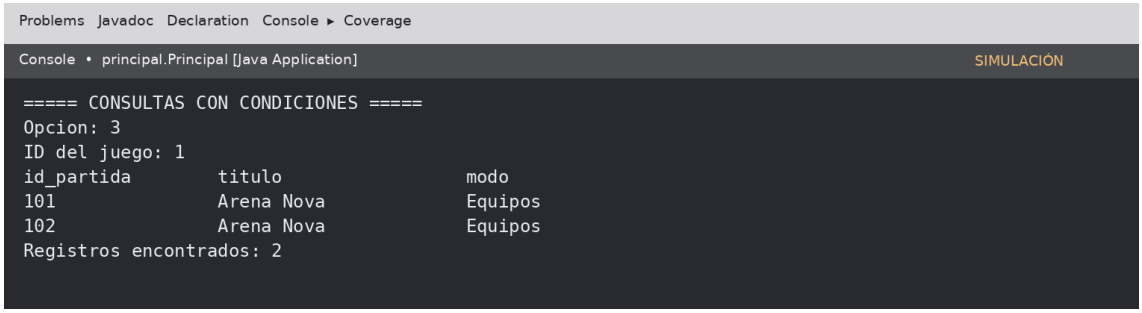


Figura A.3. Consola de Eclipse — simulación de uso relacionada con Consultas.java.

Código fuente completo:

```
package consultas;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.ResultSetMetaData;
import java.sql.SQLException;
import java.util.Scanner;

import conexion.BaseDeDatos;
import principal.Principal;

public class Consultas {
    public static void menu(Scanner scanner) throws SQLException
    {
        int opcion;
        do
        {
            System.out.println("\n===== CONSULTAS CON CONDICIONES =====");
            System.out.println("1. Usuarios activos");
            System.out.println("2. Juegos por capacidad minima");
            System.out.println("3. Partidas de un juego");
            System.out.println("4. Historial de un usuario");
            System.out.println("5. Jugadores con puntaje minimo");
            System.out.println("6. Partidas finalizadas por region");
            System.out.println("7. Logros de un usuario");
            System.out.println("0. Volver");
            System.out.print("Opcion: ");
            opcion = Principal.leerNumero(scanner);
            switch(opcion)
            {
                case 1 -> ejecutar("SELECT id_usuario, nombre_usuario, estado FROM usuario WHERE
estado = ? ORDER BY nombre_usuario", "ACTIVO");
                case 2 -> {
                    System.out.print("Capacidad minima: ");
                    ejecutar("SELECT id_juego, titulo, max_jugadores FROM juego WHERE max_jugadores
>= ? ORDER BY max_jugadores DESC", Principal.leerNumero(scanner));
                }
                case 3 -> {
                    System.out.print("ID del juego: ");
                    ejecutar("SELECT p.id_partida, j.titulo, p.modo, p.estado FROM partida p JOIN
juego j ON p.id_juego = j.id_juego WHERE j.id_juego = ? ORDER BY p.id_partida",
Principal.leerNumero(scanner));
                }
                case 4 -> {
                    System.out.print("ID del usuario: ");
```

```
                    ejecutar("SELECT u.nombre_usuario, j.titulo, p.id_partida, pa.puntos,
pa.resultado FROM participacion pa JOIN usuario u ON pa.id_usuario = u.id_usuario JOIN partida p ON
pa.id_partida = p.id_partida JOIN juego j ON p.id_juego = j.id_juego WHERE u.id_usuario = ? ORDER BY
p.inicio DESC", Principal.leerNumero(scanner));
                }
                case 5 -> {
                    System.out.print("Puntaje minimo: ");
                    ejecutar("SELECT u.nombre_usuario, pa.id_partida, pa.puntos, pa.bajas FROM
participacion pa JOIN usuario u ON pa.id_usuario = u.id_usuario WHERE pa.puntos >= ? ORDER BY
pa.puntos DESC", Principal.leerNumero(scanner));
                }
                case 6 -> {
                    System.out.print("Region (LATAM, EU o NA): ");
                    ejecutar("SELECT p.id_partida, j.titulo, p.region, p.fin FROM partida p JOIN
juego j ON p.id_juego = j.id_juego WHERE p.estado = ? AND p.region = ? ORDER BY p.fin DESC",
"FINALIZADA", scanner.nextLine().trim());
                }
                case 7 -> {
                    System.out.print("ID del usuario: ");
                    ejecutar("SELECT u.nombre_usuario, l.titulo, j.titulo AS juego,
ld.fecha_desbloqueo FROM logro_desbloqueado ld JOIN usuario u ON ld.id_usuario = u.id_usuario JOIN
logro l ON ld.id_logro = l.id_logro JOIN juego j ON l.id_juego = j.id_juego WHERE u.id_usuario = ?
ORDER BY ld.fecha_desbloqueo", Principal.leerNumero(scanner));
                }
                case 0 -> { }
                default -> System.out.println("Opcion invalida.");
            }
        } while(opcion != 0);
    }

    public static void ejecutar(String sql, Object... parametros) throws SQLException
    {
        try(Connection con = BaseDeDatos.getConnection(); PreparedStatement ps =
con.prepareStatement(sql))
        {
            for(int i = 0; i < parametros.length; i++)
            {
                ps.setObject(i + 1, parametros[i]);
            }
            try(ResultSet rs = ps.executeQuery())
            {
                ResultSetMetaData meta = rs.getMetaData();
                int columnas = meta.getColumnCount();
                int filas = 0;
                for(int i = 1; i <= columnas; i++)
                {
                    System.out.printf("%-23s", meta.getColumnLabel(i));
                }
                System.out.println();
                while(rs.next())
                {
                    for(int i = 1; i <= columnas; i++)
                    {
                        String valor = rs.getString(i);
                        System.out.printf("%-23.23s", valor == null ? "NULL" : valor);
                    }
                    System.out.println();
                    filas++;
                }
                System.out.println("Registros encontrados: " + filas);
            }
        }
    }
}
```

A.4. consultas/EjerciciosSQL.java

Realiza tres sentencias UPDATE y tres DELETE con filtros concretos, mediante PreparedStatement. Las seis operaciones se confirman juntas con una clave o se revierten con rollback, sin eliminar registros padre referenciados.

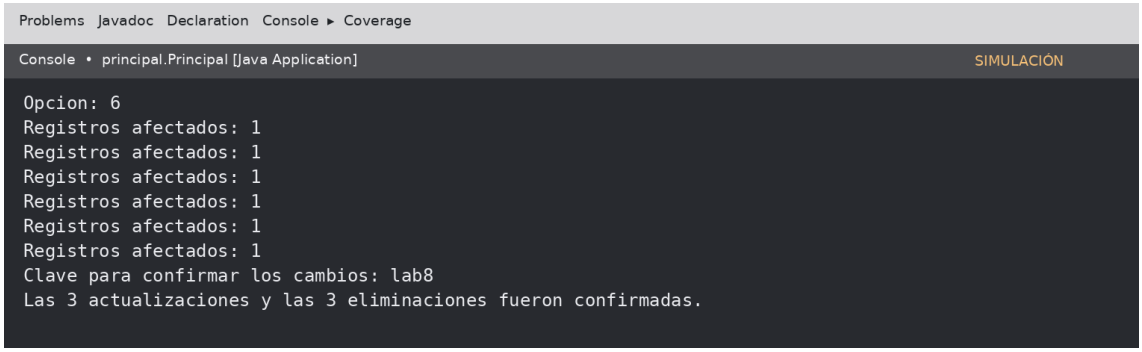


Figura A.4. Consola de Eclipse — simulación de uso relacionada con EjerciciosSQL.java.

Código fuente completo:

```
package consultas;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.SQLException;
import java.util.Scanner;

import conexion.BaseDeDatos;
import seguridad.Seguridad;

public class EjerciciosSQL {
    public static void aplicarCambios(Scanner scanner) throws SQLException
    {
        try(Connection con = BaseDeDatos.getConnection())
        {
            con.setAutoCommit(false);
            try
            {
                modificar(con, "UPDATE usuario SET estado = ? WHERE id_usuario = ?", "INACTIVO", 10);
                modificar(con, "UPDATE juego SET version_actual = ? WHERE id_juego = ?", "1.3", 1);
                modificar(con, "UPDATE perfil_jugador SET pais = ? WHERE id_usuario = ?", "Bolivia",
5);

                modificar(con, "DELETE FROM logro_desbloqueado WHERE id_usuario = ? AND id_logro
= ?", 9, 9);
                modificar(con, "DELETE FROM participacion WHERE id_usuario = ? AND id_partida = ?",
9, 109);
                modificar(con, "DELETE FROM estadistica_juego WHERE id_usuario = ? AND id_juego = ?",
10, 5);

                if(Seguridad.confirmar(scanner))
                {
                    con.commit();
                    System.out.println("Las 3 actualizaciones y las 3 eliminaciones fueron
confirmadas.");
                }
                else
                {
                    con.rollback();
                    System.out.println("Clave incorrecta: ninguna de las seis operaciones se
guardo.");
                }
            }
            catch(SQLException e)
            {
```

```
                con.rollback();
                throw e;
            }
        }
    }

    private static void modificar(Connection con, String sql, Object... parametros) throws
SQLException
    {
        try(PreparedStatement ps = con.prepareStatement(sql))
        {
            for(int i = 0; i < parametros.length; i++)
            {
                ps.setObject(i + 1, parametros[i]);
            }
            System.out.println("Registros afectados: " + ps.executeUpdate());
        }
    }
}
```

A.5. gestor/GestorCRUD.java

Permite insertar, mostrar, modificar y eliminar información de todas las entidades. Obtiene los atributos con PRAGMA table_info, admite claves primarias compuestas y solo acepta nombres de tabla de una lista controlada. Los cambios requieren clave y se ejecutan con transacciones.

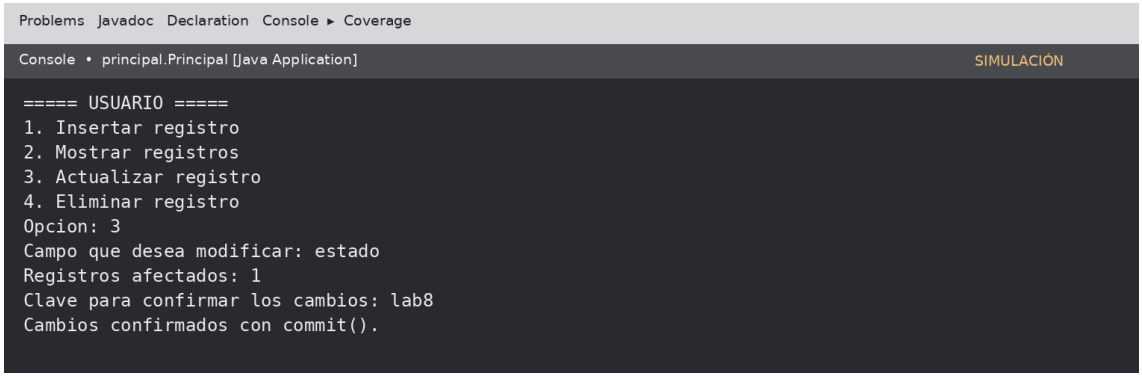


Figura A.5. Consola de Eclipse — simulación de uso relacionada con GestorCRUD.java.

Código fuente completo:

```
package gestor;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

import conexion.BaseDeDatos;
import principal.Principal;
import seguridad.Seguridad;

public class GestorCRUD {
    public static final String[] TABLAS = {
            "usuario", "perfil_jugador", "juego", "partida", "participacion",
            "estadistica_juego", "logro", "logro_desbloqueado"
    };

    private record Campo(String nombre, String tipo, boolean obligatorio, int clave, String defecto)
{ }

    public static boolean esTablaValida(String tabla)
    {
        for(String nombre : TABLAS)
        {
            if(nombre.equals(tabla))
            {
                return true;
            }
        }
        return false;
    }

    public static void menu(Scanner scanner) throws SQLException
    {
        System.out.println("\n===== TABLAS DISPONIBLES =====");
        for(int i = 0; i < TABLAS.length; i++)
        {
            System.out.println((i + 1) + ". " + TABLAS[i]);
        }
        System.out.print("Seleccione una tabla: ");
```

```
        int indice = Principal.leerNumero(scanner) - 1;
        if(indice < 0 || indice >= TABLAS.length)
        {
            System.out.println("Tabla invalida.");
            return;
        }
        String tabla = TABLAS[indice];
        int opcion;
        do
        {
            System.out.println("\n===== " + tabla.toUpperCase() + " =====");
            System.out.println("1. Insertar registro");
            System.out.println("2. Mostrar registros");
            System.out.println("3. Actualizar registro");
            System.out.println("4. Eliminar registro");
            System.out.println("0. Volver");
            System.out.print("Opcion: ");
            opcion = Principal.leerNumero(scanner);
            switch(opcion)
            {
                case 1 -> insertar(tabla, scanner);
                case 2 -> mostrar(tabla);
                case 3 -> actualizar(tabla, scanner);
                case 4 -> eliminar(tabla, scanner);
                case 0 -> { }
                default -> System.out.println("Opcion invalida.");
            }
        } while(opcion != 0);
    }

    private static List<Campo> obtenerCampos(Connection con, String tabla) throws SQLException
    {
        List<Campo> campos = new ArrayList<>();
        try(Statement stmt = con.createStatement(); ResultSet rs = stmt.executeQuery("PRAGMA
table_info(" + tabla + ")"))
        {
            while(rs.next())
            {
                campos.add(new Campo(rs.getString("name"), rs.getString("type"), rs.getInt("notnull")
!= 0,
                        rs.getInt("pk"), rs.getString("dflt_value")));
            }
        }
        return campos;
    }

    private static Object pedirDato(Scanner scanner, Campo campo, boolean admiteVacio)
    {
        while(true)
        {
            System.out.print(campo.nombre() + " (" + campo.tipo() + ")" + (admiteVacio ? " [vacio =
NULL o valor predeterminado]" : "") + ": ");
            String valor = scanner.nextLine().trim();
            if(valor.isEmpty())
            {
                if(admiteVacio)
                {
                    return null;
                }
                System.out.println("El campo es obligatorio.");
                continue;
            }
            if(campo.tipo().toUpperCase().contains("INT"))
            {
                try
                {
                    return Integer.parseInt(valor);
                }
                catch(NumberFormatException e)
                {
                    System.out.println("Debe ingresar un numero entero.");
                    continue;
                }
            }
            return valor;
```

```
        }
    }

    private static void insertar(String tabla, Scanner scanner) throws SQLException
    {
        try(Connection con = BaseDeDatos.getConnection())
        {
            List<Campo> campos = obtenerCampos(con, tabla);
            List<String> nombres = new ArrayList<>();
            List<Object> valores = new ArrayList<>();
            for(Campo campo : campos)
            {
                boolean claveAutomatica = (tabla.equals("usuario") || tabla.equals("juego") ||
tabla.equals("partida") || tabla.equals("logro")) && campo.clave() == 1 &&
campo.tipo().equalsIgnoreCase("INTEGER");
                boolean opcional = campo.defecto() != null || !campo.obligatorio() && campo.clave()
== 0 || claveAutomatica;
                Object valor = pedirDato(scanner, campo, opcional);
                if(valor != null)
                {
                    nombres.add(campo.nombre());
                    valores.add(valor);
                }
            }
            String sql;
            if(nombres.isEmpty())
            {
                sql = "INSERT INTO " + tabla + " DEFAULT VALUES";
            }
            else
            {
                sql = "INSERT INTO " + tabla + " (" + String.join(", ", nombres) + ") VALUES (" +
String.join(", ", java.util.Collections.nCopies(nombres.size(), "?")) + ")";
            }
            con.setAutoCommit(false);
            if(tabla.equals("participacion") && !puedeIngresar(con, nombres, valores))
            {
                con.rollback();
                System.out.println("El usuario ya participa en otra partida activa.");
                return;
            }
            try(PreparedStatement ps = con.prepareStatement(sql))
            {
                for(int i = 0; i < valores.size(); i++)
                {
                    ps.setObject(i + 1, valores.get(i));
                }
                confirmarCambio(con, ps, scanner);
            }
            catch(SQLException e)
            {
                con.rollback();
                System.out.println("No se pudo insertar: " + e.getMessage());
            }
        }
    }

    private static void mostrar(String tabla) throws SQLException
    {
        try(Connection con = BaseDeDatos.getConnection(); Statement stmt = con.createStatement();
ResultSet rs = stmt.executeQuery("SELECT * FROM " + tabla + " LIMIT 100"))
        {
            var meta = rs.getMetaData();
            int columnas = meta.getColumnCount();
            for(int i = 1; i <= columnas; i++)
            {
                System.out.printf("%-20.20s", meta.getColumnName(i));
            }
            System.out.println();
            int filas = 0;
            while(rs.next())
            {
                for(int i = 1; i <= columnas; i++)
                {
                    Object valor = rs.getObject(i);
```

```
                    System.out.printf("%-20.20s", valor == null ? "NULL" : valor.toString());
                }
                System.out.println();
                filas++;
            }
            System.out.println("Registros mostrados: " + filas);
        }
    }

    private static List<Campo> claves(List<Campo> campos) throws SQLException
    {
        List<Campo> claves = campos.stream().filter(c -> c.clave() > 0).sorted((a, b) ->
Integer.compare(a.clave(), b.clave())).toList();
        if(claves.isEmpty())
        {
            throw new SQLException("Esta tabla no tiene una clave primaria definida.");
        }
        return claves;
    }

    private static List<Object> pedirClaves(Scanner scanner, List<Campo> claves)
    {
        List<Object> valores = new ArrayList<>();
        System.out.println("Identifique el registro por su clave primaria:");
        for(Campo clave : claves)
        {
            valores.add(pedirDato(scanner, clave, false));
        }
        return valores;
    }

    private static String condicion(List<Campo> claves)
    {
        return String.join(" AND ", claves.stream().map(c -> c.nombre() + " = ?").toList());
    }

    private static void actualizar(String tabla, Scanner scanner) throws SQLException
    {
        try(Connection con = BaseDeDatos.getConnection())
        {
            List<Campo> campos = obtenerCampos(con, tabla);
            List<Campo> claves = claves(campos);
            System.out.println("Campos modificables:");
            for(Campo campo : campos)
            {
                if(campo.clave() == 0)
                {
                    System.out.println("- " + campo.nombre());
                }
            }
            System.out.print("Campo que desea modificar: ");
            String nombre = scanner.nextLine().trim();
            Campo seleccionado = null;
            for(Campo campo : campos)
            {
                if(campo.nombre().equals(nombre) && campo.clave() == 0)
                {
                    seleccionado = campo;
                }
            }
            if(seleccionado == null)
            {
                System.out.println("Campo invalido.");
                return;
            }
            Object nuevoValor = pedirDato(scanner, seleccionado, !seleccionado.obligatorio());
            List<Object> ids = pedirClaves(scanner, claves);
            String sql = "UPDATE " + tabla + " SET " + nombre + " = ? WHERE " + condicion(claves);
            con.setAutoCommit(false);
            try(PreparedStatement ps = con.prepareStatement(sql))
            {
                ps.setObject(1, nuevoValor);
                for(int i = 0; i < ids.size(); i++)
                {
                    ps.setObject(i + 2, ids.get(i));
```

```
                }
                confirmarCambio(con, ps, scanner);
            }
            catch(SQLException e)
            {
                con.rollback();
                System.out.println("No se pudo actualizar: " + e.getMessage());
            }
        }
    }

    private static void eliminar(String tabla, Scanner scanner) throws SQLException
    {
        try(Connection con = BaseDeDatos.getConnection())
        {
            List<Campo> claves = claves(obtenerCampos(con, tabla));
            List<Object> ids = pedirClaves(scanner, claves);
            String sql = "DELETE FROM " + tabla + " WHERE " + condicion(claves);
            con.setAutoCommit(false);
            try(PreparedStatement ps = con.prepareStatement(sql))
            {
                for(int i = 0; i < ids.size(); i++)
                {
                    ps.setObject(i + 1, ids.get(i));
                }
                confirmarCambio(con, ps, scanner);
            }
            catch(SQLException e)
            {
                con.rollback();
                System.out.println("No se pudo eliminar: " + e.getMessage());
            }
        }
    }

    private static boolean puedeIngresar(Connection con, List<String> nombres, List<Object> valores)
throws SQLException
    {
        if(!nombres.contains("id_usuario") || !nombres.contains("id_partida"))
        {
            return false;
        }
        int idUsuario = ((Number) valores.get(nombres.indexOf("id_usuario"))).intValue();
        int idPartida = ((Number) valores.get(nombres.indexOf("id_partida"))).intValue();
        try(PreparedStatement ps = con.prepareStatement("SELECT estado FROM partida WHERE id_partida
= ?"))
        {
            ps.setInt(1, idPartida);
            try(ResultSet rs = ps.executeQuery())
            {
                if(!rs.next())
                {
                    return false;
                }
                if(!"EN_CURSO".equalsIgnoreCase(rs.getString(1)))
                {
                    return true;
                }
            }
        }
        String sql = "SELECT COUNT(*) FROM participacion pa JOIN partida p ON pa.id_partida =
p.id_partida WHERE pa.id_usuario = ? AND pa.id_partida <> ? AND p.estado = ?";
        try(PreparedStatement ps = con.prepareStatement(sql))
        {
            ps.setInt(1, idUsuario);
            ps.setInt(2, idPartida);
            ps.setString(3, "EN_CURSO");
            try(ResultSet rs = ps.executeQuery())
            {
                return rs.next() && rs.getInt(1) == 0;
            }
        }
    }
```

```
    private static void confirmarCambio(Connection con, PreparedStatement ps, Scanner scanner) throws
SQLException
    {
        int filas = ps.executeUpdate();
        System.out.println("Registros afectados: " + filas);
        if(Seguridad.confirmar(scanner))
        {
            con.commit();
            System.out.println("Cambios confirmados con commit().");
        }
        else
        {
            con.rollback();
            System.out.println("Clave incorrecta. Cambios revertidos con rollback().");
        }
    }
}
```

A.6. gestor/GestorRegistros.java

Obtiene filas SQLite, las convierte en un arreglo Registro[] y realiza los filtros en Java, sin delegar la lógica de consulta a SELECT. El usuario decide columnas, condición, orden ascendente o descendente y límite.

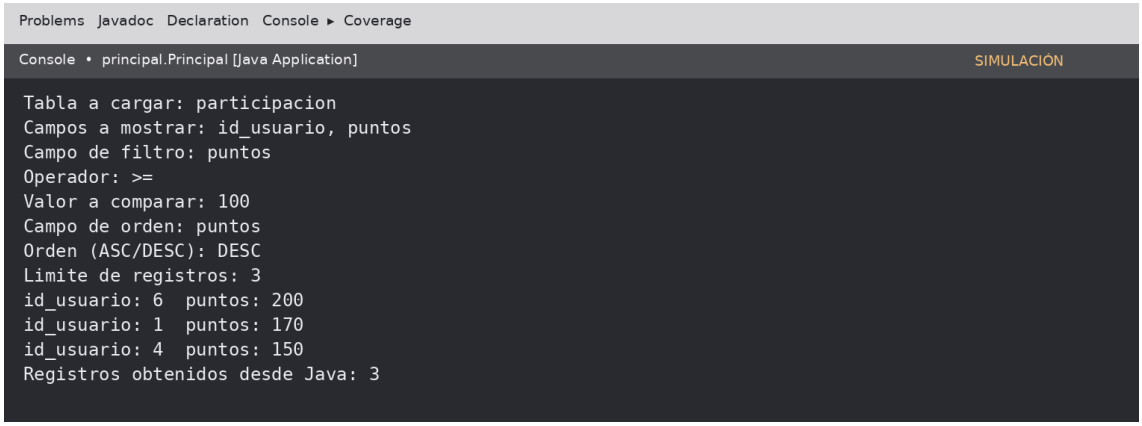


Figura A.6. Consola de Eclipse — simulación de uso relacionada con GestorRegistros.java.

Código fuente completo:

```
package gestor;

import java.sql.Connection;
import java.sql.ResultSet;
import java.sql.ResultSetMetaData;
import java.sql.SQLException;
import java.sql.Statement;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Comparator;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.Scanner;

import conexion.BaseDeDatos;
import modelo.Registro;
import principal.Principal;

public class GestorRegistros {
    private Registro[] registros;

    public GestorRegistros()
    {
        registros = new Registro[0];
    }

    public void cargar(String tabla) throws SQLException
    {
        if(!GestorCRUD.esTablaValida(tabla))
        {
            throw new SQLException("Tabla no permitida.");
        }
        List<Registro> lista = new ArrayList<>();
        try(Connection con = BaseDeDatos.getConnection(); Statement stmt = con.createStatement();
ResultSet rs = stmt.executeQuery("SELECT * FROM " + tabla))
        {
            ResultSetMetaData meta = rs.getMetaData();
            while(rs.next())
            {
                Map<String, Object> campos = new LinkedHashMap<>();
                for(int i = 1; i <= meta.getColumnCount(); i++)
```

```
                {
                    campos.put(meta.getColumnName(i), rs.getObject(i));
                }
                lista.add(new Registro(campos));
            }
        }
        registros = lista.toArray(new Registro[0]);
    }

    public Registro[] consultar(String[] campos, String campoFiltro, String operador, String valor,
String campoOrden, boolean ascendente, int limite)
    {
        List<Registro> encontrados = new ArrayList<>();
        for(Registro registro : registros)
        {
            if(campoFiltro.isBlank() || cumple(registro.get(campoFiltro), operador, valor))
            {
                encontrados.add(registro);
            }
        }
        if(!campoOrden.isBlank())
        {
            Comparator<Registro> comparador = (a, b) -> comparar(a.get(campoOrden),
b.get(campoOrden));
            encontrados.sort(ascendente ? comparador : comparador.reversed());
        }
        if(limite > 0 && encontrados.size() > limite)
        {
            encontrados = new ArrayList<>(encontrados.subList(0, limite));
        }
        for(Registro registro : encontrados)
        {
            for(String campo : campos)
            {
                System.out.print(campo + ": " + registro.get(campo) + "  ");
            }
            System.out.println();
        }
        System.out.println("Registros obtenidos desde Java: " + encontrados.size());
        return encontrados.toArray(new Registro[0]);
    }

    private boolean cumple(Object actual, String operador, String valor)
    {
        if(actual == null)
        {
            return operador.equals("=") && valor.equalsIgnoreCase("NULL");
        }
        int comparacion = comparar(actual, valor);
        return switch(operador)
        {
            case "=" -> comparacion == 0;
            case "!=" -> comparacion != 0;
            case ">" -> comparacion > 0;
            case ">=" -> comparacion >= 0;
            case "<" -> comparacion < 0;
            case "<=" -> comparacion <= 0;
            case "contiene" -> actual.toString().toLowerCase().contains(valor.toLowerCase());
            default -> false;
        };
    }

    private int comparar(Object a, Object b)
    {
        if(a == null && b == null) return 0;
        if(a == null) return -1;
        if(b == null) return 1;
        try
        {
            return Double.compare(Double.parseDouble(a.toString()),
Double.parseDouble(b.toString()));
        }
        catch(NumberFormatException e)
        {
            return a.toString().compareToIgnoreCase(b.toString());
```

```
        }
    }

    public static void menu(Scanner scanner) throws SQLException
    {
        System.out.println("\nTablas disponibles: " + String.join(", ", GestorCRUD.TABLAS));
        System.out.print("Tabla a cargar: ");
        String tabla = scanner.nextLine().trim().toLowerCase();
        if(!GestorCRUD.esTablaValida(tabla))
        {
            System.out.println("Tabla invalida.");
            return;
        }
        GestorRegistros gestor = new GestorRegistros();
        gestor.cargar(tabla);
        if(gestor.registros.length == 0)
        {
            System.out.println("No hay registros en esa tabla.");
            return;
        }
        String[] disponibles = gestor.registros[0].obtenerCampos();
        System.out.println("Campos: " + String.join(", ", disponibles));
        System.out.print("Campos a mostrar (separados por coma o *): ");
        String entrada = scanner.nextLine().trim();
        String[] campos = entrada.equals("*") ? disponibles :
Arrays.stream(entrada.split(",")).map(String::trim).toArray(String[]::new);
        for(String campo : campos)
        {
            if(!gestor.registros[0].contiene(campo))
            {
                System.out.println("Campo no valido: " + campo);
                return;
            }
        }
        System.out.print("Campo de filtro (vacio = ninguno): ");
        String campoFiltro = scanner.nextLine().trim();
        if(!campoFiltro.isEmpty() && !gestor.registros[0].contiene(campoFiltro))
        {
            System.out.println("Campo de filtro invalido.");
            return;
        }
        String operador = "=";
        String valor = "";
        if(!campoFiltro.isEmpty())
        {
            System.out.print("Operador (=, !=, >, >=, <, <=, contiene): ");
            operador = scanner.nextLine().trim();
            if(!Arrays.asList("=", "!=", ">", ">=", "<", "<=", "contiene").contains(operador))
            {
                System.out.println("Operador invalido.");
                return;
            }
            System.out.print("Valor a comparar: ");
            valor = scanner.nextLine();
        }
        System.out.print("Campo de orden (vacio = ninguno): ");
        String campoOrden = scanner.nextLine().trim();
        if(!campoOrden.isEmpty() && !gestor.registros[0].contiene(campoOrden))
        {
            System.out.println("Campo de orden invalido.");
            return;
        }
        System.out.print("Orden (ASC/DESC): ");
        boolean ascendente = !scanner.nextLine().trim().equalsIgnoreCase("DESC");
        System.out.print("Limite de registros (0 = todos): ");
        int limite = Principal.leerNumero(scanner);
        gestor.consultar(campos, campoFiltro, operador, valor, campoOrden, ascendente, limite);
    }
}
```

A.7. modelo/Registro.java

Representa una fila de una tabla mediante un objeto Java con atributos identificados por nombre. Facilita que la clase gestora pueda consultar tablas distintas sin escribir un modelo específico por cada una.

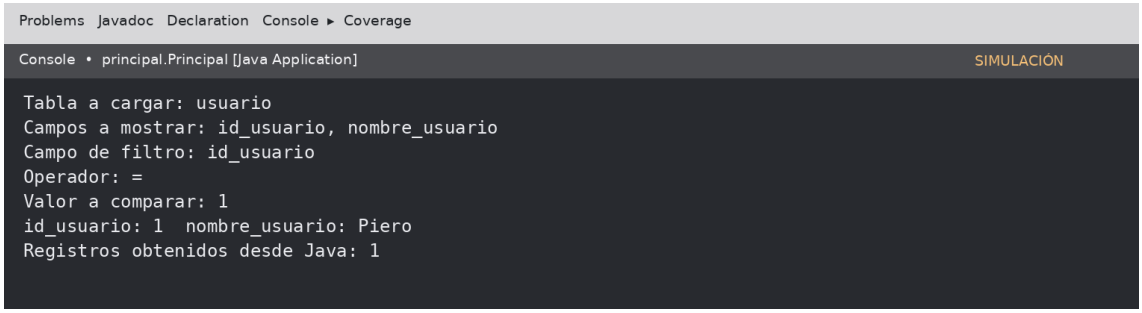


Figura A.7. Consola de Eclipse — simulación de uso relacionada con Registro.java.

Código fuente completo:

```
package modelo;

import java.util.LinkedHashMap;
import java.util.Map;

public class Registro {
    private final Map<String, Object> campos;

    public Registro(Map<String, Object> campos)
    {
        this.campos = new LinkedHashMap<>(campos);
    }

    public Object get(String nombre)
    {
        return campos.get(nombre);
    }

    public boolean contiene(String nombre)
    {
        return campos.containsKey(nombre);
    }

    public String[] obtenerCampos()
    {
        return campos.keySet().toArray(new String[0]);
    }
}
```

A.8. principal/Principal.java

Inicializa la base de datos al iniciar, muestra las opciones de ejecución y conecta las operaciones de carga, consultas, CRUD, gestor de objetos y transacciones. Se mantiene una estructura clásica de main, switch y métodos cortos.

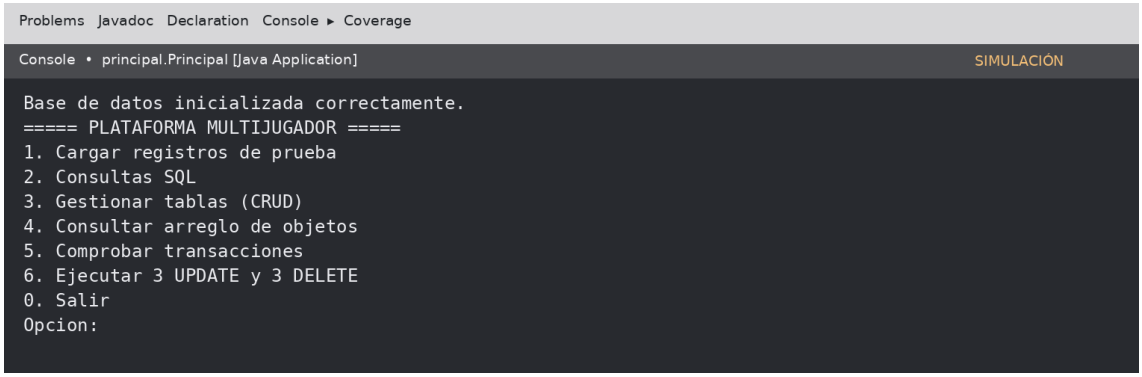


Figura A.8. Consola de Eclipse — simulación de uso relacionada con Principal.java.

Código fuente completo:

```
package principal;

import java.sql.Connection;
import java.sql.SQLException;
import java.util.Scanner;

import conexion.BaseDeDatos;
import conexion.DatosPrueba;
import consultas.Consultas;
import consultas.EjerciciosSQL;
import gestor.GestorCRUD;
import gestor.GestorRegistros;
import seguridad.Seguridad;

public class Principal {
    public static void main(String[] args)
    {
        try(Scanner scanner = new Scanner(System.in))
        {
            BaseDeDatos.inicializar();
            System.out.println("Base de datos inicializada correctamente.");
            int opcion;

            do
            {
                System.out.println("\n===== PLATAFORMA MULTIJUGADOR =====");
                System.out.println("1. Cargar registros de prueba");
                System.out.println("2. Consultas SQL");
                System.out.println("3. Gestionar tablas (CRUD)");
                System.out.println("4. Consultar arreglo de objetos");
                System.out.println("5. Comprobar transacciones");
                System.out.println("6. Ejecutar 3 UPDATE y 3 DELETE");
                System.out.println("0. Salir");
                System.out.print("Opcion: ");
                opcion = leerNumero(scanner);

                try
                {
                    switch(opcion)
                    {
                        case 1 -> cargarDatos(scanner);
                        case 2 -> Consultas.menu(scanner);
                        case 3 -> GestorCRUD.menu(scanner);
                        case 4 -> GestorRegistros.menu(scanner);
```

```
                    case 5 -> comprobarTransaccion(scanner);
                    case 6 -> EjerciciosSQL.aplicarCambios(scanner);
                    case 0 -> System.out.println("Hasta luego.");
                    default -> System.out.println("Opcion invalida.");
                }
            }
            catch(SQLException e)
            {
                System.out.println("Error de base de datos: " + e.getMessage());
            }
        } while(opcion != 0);
    }
    catch(SQLException e)
    {
        System.out.println("No se pudo iniciar: " + e.getMessage());
    }
}

public static int leerNumero(Scanner scanner)
{
    try
    {
        return Integer.parseInt(scanner.nextLine().trim());
    }
    catch(NumberFormatException e)
    {
        return -1;
    }
}

private static void cargarDatos(Scanner scanner) throws SQLException
{
    try(Connection con = BaseDeDatos.getConnection())
    {
        con.setAutoCommit(false);
        try
        {
            DatosPrueba.cargar(con);
            if(Seguridad.confirmar(scanner))
            {
                con.commit();
                System.out.println("Carga confirmada: 10 registros por tabla como minimo.");
            }
            else
            {
                con.rollback();
                System.out.println("Clave incorrecta. Se revirtio la carga.");
            }
        }
        catch(SQLException e)
        {
            con.rollback();
            throw e;
        }
    }
}

private static void comprobarTransaccion(Scanner scanner) throws SQLException
{
    try(Connection con = BaseDeDatos.getConnection())
    {
        con.setAutoCommit(false);
        try
        {
            String sql = "UPDATE usuario SET estado = estado WHERE id_usuario = 1";
            try(var ps = con.prepareStatement(sql))
            {
                int filas = ps.executeUpdate();
                System.out.println("Registros afectados antes de confirmar: " + filas);
            }
            if(Seguridad.confirmar(scanner))
            {
                con.commit();
                System.out.println("Transaccion confirmada con commit().");
            }
```

```
                else
                {
                    con.rollback();
                    System.out.println("Transaccion revertida con rollback().");
                }
            }
            catch(SQLException e)
            {
                con.rollback();
                throw e;
            }
        }
    }
}
```

A.9. seguridad/Seguridad.java

Solicita la clave antes de confirmar operaciones que alteran datos. Se permite configurar LAB8_CLAVE o utilizar la clave académica lab8; esta forma de confirmación es didáctica y no sustituye la autenticación real de un sistema en producción.

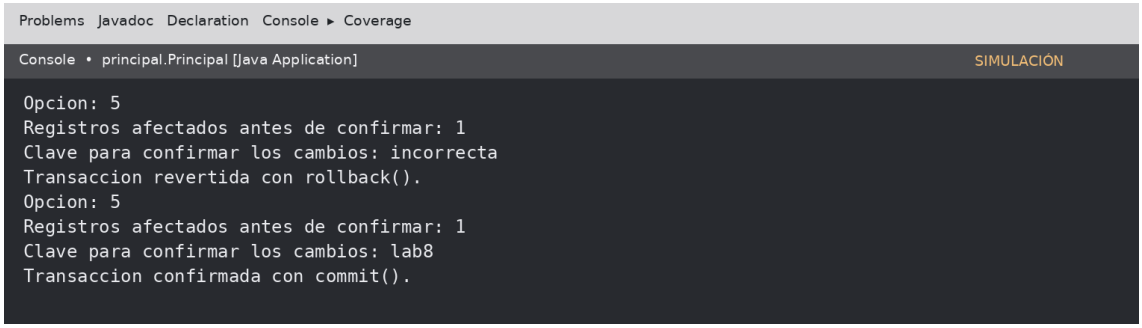


Figura A.9. Consola de Eclipse — simulación de uso relacionada con Seguridad.java.

Código fuente completo:

```
package seguridad;

import java.util.Scanner;

public class Seguridad {
    public static boolean confirmar(Scanner scanner)
    {
        String clave = System.getenv("LAB8_CLAVE");
        if(clave == null || clave.isBlank())
        {
            clave = "lab8";
        }
        System.out.print("Clave para confirmar los cambios: ");
        String ingresada = scanner.nextLine();
        return ingresada.equals(clave);
    }
}
```

ANEXO COMPLEMENTARIO. CONFIGURACIÓN DE ECLIPSE

El proyecto original conserva ejemplo/.project, ejemplo/.classpath, sus preferencias de Eclipse y ejemplo/src/module-info.java. Se respetaron para no alterar el trabajo existente. La referencia al archivo sqlite-jdbc dentro de .classpath apunta a una ruta absoluta del equipo del autor y debe corregirse al abrir el proyecto desde otro equipo.

El archivo module-info.java declara requires java.sql, necesario para que las clases Connection, Statement y PreparedStatement sean accesibles desde el proyecto modular. La disponibilidad del controlador SQLite se configura a través de las bibliotecas del proyecto.